Day 11 - Complete Lecture Notes

React: Class and Function Components, Props, State, Virtual DOM, Hooks (useState), Lifecycle, Data Fetching, List Rendering and Conditional Rendering

Session: React class and function components with props and state (31 October) | **Instructor:** Abhishek Purohit | **Course:** React JS

How to read these notes. The material follows the order in which the instructor taught it. Boxes with a **yellow** background are important points or warnings from the lecture, boxes with a **blue** background are extra explanations added by me to help you understand (they are not from the lecture), **red** boxes are common mistakes, and **green** boxes show the expected output.

Contents

Part	Topic
1	Project setup with Vite and cleaning the boilerplate
2	Components: class components (Navbar, Footer), tags, closing rule
3	JSX: what it is, XML vs HTML, why every tag must be closed
4	Function components (NavigationBar), class vs function, single parent rule
5	Features of a component: props and state
6	Props: reusable Intro component, props object, destructuring, read-only rule
7	Props in a class component (this.props)
8	State in a class component (Scoreboard), events, setState, re-rendering
9	Virtual DOM and why it is faster
10	Hooks and state in a function component (useState)
11	Component life cycle: mounting, unmounting, constructor, render, componentDidMount
12	Fetching data in componentDidMount (UsersList) and holding it in state
13	List rendering with map(), table rows, conditional rendering (loading message)
14	End of session: MCP discussion and human verification
15	Revision: summary, definitions, differences, mistakes, interview Q&A, exercises, checklist

Part 1: Creating the React Application (Vite) and Cleaning the Boilerplate

The instructor began by checking that everyone had created and started a React application. A new folder named after the date (31 October) was created, and a fresh React application was created inside it so that all the remaining concepts could be demonstrated on it.

1.1 Creating the project

Open the folder in the terminal and run the Vite command. Vite (pronounced like 'veet') is a build tool that creates the starting React project (the **boilerplate**) for you.

```
npm create vite@latest
```

The command asks a few questions. The choices made in the lecture were:

Prompt	Choice in the lecture	Meaning
Project name	react-demo (typed as 'React demo')	Name of the folder that Vite creates. Use lowercase letters and hyphens.

Prompt	Choice in the lecture	Meaning
Framework	React	We are building a React project.
Variant	JavaScript	Plain JavaScript (not TypeScript). The instructor also chose the ESLint option.
Install and start now?	No - done manually	The instructor chose to install dependencies and start the app by himself.

Vite then generated the boilerplate code. Open the generated folder (here, `react-demo` inside the 31 October folder) in VS Code using **File > Open Folder**.

1.2 Installing dependencies and starting the app

```
npm i          # installs all required dependencies, creates the node_modules folder
npm run dev    # starts the application on the developer machine
```

- `npm i` (short for `npm install`) reads `package.json` and downloads every library the project needs into a new **node_modules** folder. It also produces the package-lock file.
- `npm run dev` starts the development server. The app was available on port **5173**, so you open `http://localhost:5173` in the browser.
- The instructor mentioned the install was slow because of his network; it can take time depending on the internet connection.

1.3 The App component and cleaning the default code

As discussed in the previous session, the first component that React renders on the screen is the **App component**. To change what the application shows, open the `src` folder and edit **App.jsx**. The Vite template contains a lot of default code (logos, a counter, etc.). The instructor removed all of it and wrote a minimal structure.

Cleaned App.jsx

```
// src/App.jsx  (after cleaning)
function App() {
  return (
    <div>
      <h1>Welcome to our app</h1>
    </div>
  );
}

export default App;
```

- Remove the unused imports at the top (the logo images, `useState` and the CSS file). Unused code is not needed for our learning.
- Vite also generated CSS: `App.css` and `index.css`. Another CSS import exists in **main.jsx**. The instructor removed these so that we work with a simple, plain React application without any predefined styling.
- Result: App.jsx simply displays a `div`, and inside it an `h1`.

Expected output

```
Welcome to our app
```

> Important: after you remove a CSS file, also remove the line that imports it (for example `import './index.css'` in main.jsx). If the file is deleted but the import remains, the app shows an error.

Part 2: Class Components

In the previous session the instructor explained that there are two ways of creating your own component: a **class component** and a **function component**. This session continues with the class component first, and then covers the function component.

2.1 Creating the Navbar class component

Inside the `src` folder, create a new file named **Navbar.jsx** (the extension `.jsx` is explained in Part 3).

Phase 1: A normal class is not a component

The instructor first stated the problem: if you only write a class named `Navbar`, it is a **normal, regular class**. It will not behave as a React component (a UI element), because it does not have the properties and behaviours that a UI element needs.

```
// Not a component - just a plain JavaScript class
class Navbar {
}
```

Phase 2: Inherit from the Component class

To give the class those properties and behaviours, it must **inherit (extend) the `Component` class**. `Component` is a predefined class that the React team has given us out of the box, and it is imported from the React module. Only when your class extends `Component` is it recognised and treated as a UI element.

```
import { Component } from 'react';

class Navbar extends Component {
}
```

Phase 3: Override render() to provide the user interface

Next, we must decide what the component should look like. The `Component` class already contains a function named **render**. In our class we **override** it. The meaning of 'render' is 'to provide something'; the render function provides the user interface for the component. Whenever React needs to display the class component, it calls the `render` function behind the scenes to get the UI. Therefore, in a class component, all the HTML code goes inside `render()`.

The render function must **return** the HTML structure inside parentheses. For the navbar: one division with three anchor tags (Home, About Us, Contact Us). To avoid broken links, `href="#"` is used for now.

Complete Navbar class component

```
// src/Navbar.jsx
import { Component } from 'react';

export class Navbar extends Component {
  render() {
    return (
      <div>
        <a href="#">Home</a>
        <a href="#">About Us</a>
        <a href="#">Contact Us</a>
      </div>
    );
  }
}
```

Line-by-line explanation

Line / part	Explanation
`import { Component } from 'react'`	Brings in the predefined `Component` class from the React package.

Line / part	Explanation
`export class Navbar`	Creates the class and **exports** it so that other files (like App.jsx) can import and display it. Without export, no other file can use it.
`extends Component`	Inheritance. This is what turns an ordinary class into a React UI component.
`render() { ... }`	Overrides the render function of the parent class. React calls it automatically to get the UI.
`return ( ... )`	Returns the HTML (JSX) that should be displayed. The parentheses let us spread the HTML over multiple lines.
`<a href="#">`	Anchor tags. `href="#"` is a harmless placeholder link.

2.2 Displaying the component inside App

Just creating a component does not display it anywhere. The developer decides where it should appear. Here, the Navbar should be displayed inside the App component. This needs two things: **import** and **render**.

1. **Export** the class from Navbar.jsx (already done with `export class`).
2. **Import** it in App.jsx using an import statement. The path `./Navbar` means 'the same folder as App.jsx'. The instructor noted that you do not have to write the `.jsx` extension; that is already configured, so only the file name is needed.
3. **Render** it using its **tag**.

App.jsx displaying Navbar

```
// src/App.jsx
import { Navbar } from './Navbar';

function App() {
  return (
    <div>
      <Navbar></Navbar>
      <h1>Welcome to our app</h1>
    </div>
  );
}

export default App;
```

> **Additional explanation (not from the lecture):** Because Navbar.jsx uses `export class Navbar` (a **named export**), it is imported with curly braces: `import { Navbar } from './Navbar'`. If a file uses `export default`, the import has no braces.

Expected output

```
Home  About Us  Contact Us          <- Navbar component (links)
Welcome to our app                  <- h1 from App
```

The most important rule: every component gets a tag

The instructor stressed this as the most important and fundamental thing in React. Whenever you create a component (class or function), React gives you a **corresponding tag with the same name as the class or function**. If the class is `Navbar`, the tag is `<Navbar>`. If you create a component named `Footer`, you get `<Footer>`. If you create `Dashboard`, you get `<Dashboard>`, and for `LoginForm` the tag is `<LoginForm>`. To display a component, you simply use its tag. The instructor asked students to write this point in their notes.

What happens behind the scenes

When the Navbar tag is displayed, React makes a call to the `render` function behind the scenes, and that function provides the user interface. So the lecture summary of creating a class component is:

1. Create a class and **inherit** the `Component` class.
2. Define the **render** function.

3. **Return** the user interface from render.
4. **Export** the class.
5. **Import** it where needed and display it using its **tag**.

> **Common mistake:** Forgetting `extends Component`, forgetting the `render` function, forgetting to `export`, or forgetting to `import` in App.jsx. Any one of these means the component will not appear (or the app will show an error).

2.3 Creating the Footer class component

The instructor repeated the same steps for a second component to get practice. Create **Footer.jsx** as a class component with an `h4` that says 'This is the footer'.

```
// src/Footer.jsx
import { Component } from 'react';

export class Footer extends Component {
  render() {
    return (
      <div>
        <h4>This is the footer</h4>
      </div>
    );
  }
}
```

Then in App.jsx, import it and display it after the `h1`:

```
// src/App.jsx
import { Navbar } from './Navbar';
import { Footer } from './Footer';

function App() {
  return (
    <div>
      <Navbar></Navbar>
      <h1>Welcome to our app</h1>
      <Footer></Footer>
    </div>
  );
}

export default App;
```

Expected output

```
Home  About Us  Contact Us          <- Navbar (top)
Welcome to our app                  <- h1 inside App
This is the footer                  <- Footer
```

This is how you define any number of class components: create a class, export it, inherit `Component`, define `render`, and return the UI from it.

2.4 The closing-tag rule

Whenever you open a tag in React you must close it. In plain HTML, some tags are 'empty' tags such as `br`, `input` and `img`, which need no closing tag. You can use all of these tags in React, but you **must close them as well**. This is mandatory in React. This applies to every tag: predefined HTML tags and the custom tags of your components.

There are two ways to close a tag:

Style	Example	When to use
Explicit closing tag	`<Navbar></Navbar>`	Tags that can contain content inside them.

Style	Example	When to use
Self-closing tag (slash at the end)	`<Navbar />` `<input />` `<img />` ` `	Tags that have no content inside (input, img, br) and components with no children. Both styles give exactly the same result.

```
<Navbar></Navbar>   // works
<Navbar />          // works - same result

<input />           // correct: empty HTML tag closed with a slash
<input>             // WRONG in React: error because the tag is not closed
```

Part 3: JSX - JavaScript and XML

Look at the code of the class component. It contains an import statement (JavaScript), the class definition (JavaScript), the function definition (JavaScript) and a return statement (JavaScript). But inside that JavaScript code we write **HTML directly**. In a regular JavaScript (.js) file, writing HTML directly is not possible. The instructor asked the class and they confirmed it is not possible in a normal JS file.

The React team made it possible by introducing a new file extension: **JSX**.

3.1 What is JSX?

JSX stands for JavaScript and XML. The `.jsx` extension enables a developer to write the JavaScript code together with the HTML code, mixing both inside one file without doing any further thing. That is why every React component file is created with the extension `.jsx`. The instructor said this is an interview question for freshers: 'What is JSX?'

3.2 Why XML? HTML versus XML

There are two kinds of markup languages: HTML and XML. In both you write tags, but the purpose is different.

Point	HTML	XML
Purpose	To **display** content or a UI on a web page.	To **hold or store data** in the form of tags.
Tags	Predefined tags (h1, p, div, a ...).	No predefined tags. The developer creates **user-defined (custom) tags**.
Example use	Showing a page to the user.	Before JSON became popular, servers sent data in XML. For example a `student` tag containing `id` and `name` tags.

Behind the scenes, React uses XML to create the user-defined tags for your own components. That is how `<Navbar>`, `<Footer>` or `<Dashboard>` become valid tags. JSX = JavaScript + XML lets you mix these in one file.

> Because JSX uses XML underneath, the XML rule applies: if you open a tag, you must close it. This is why React requires every tag to be closed (Section 2.4). In other words, the rule belongs to XML, but since React uses XML behind the scene, we have to follow it when writing React components.

Interview question	Answer
What is JSX?	JavaScript and XML. A syntax extension (.jsx) that lets you write HTML-like markup directly inside JavaScript code.
Why must tags be closed in React?	JSX is based on XML, and XML requires every opened tag to be closed.
Can we use predefined HTML tags in React?	Yes, but they must also be closed (for example ` `, `<input />`).

Part 4: Function Components

The second way of creating a component is the function component. The instructor said it is very simple: you **do not create a class and do not need inheritance**. You directly create a function that returns the UI.

4.1 Creating a function component

Create the file **NavigationBar.jsx**. The function is given the same name as the file, so it is easy to recognise the file name and the component name together. Inside the function, directly return the user interface. There is no render function.

Complete function component

```
// src/NavigationBar.jsx
export function NavigationBar() {
  return (
    <div>
      <a href="#">Home</a>
      <a href="#">About Us</a>
      <a href="#">Contact Us</a>
    </div>
  );
}
```

Using it is **exactly the same** as using a class component: export it, import it, and display it with its tag.

```
// src/App.jsx
import { NavigationBar } from './NavigationBar';

function App() {
  return (
    <div>
      <NavigationBar />
      <h1>Welcome to our app</h1>
    </div>
  );
}

export default App;
```

Expected output

```
Home  About Us  Contact Us          <- same navbar, now made with a function component
Welcome to our app
```

The definition changes (class or function), but the way of using the component is identical: you always use its tag. The self-closing form (`<NavigationBar />`) can be used because we do not put content inside.

4.2 Class component vs function component

Point	Class component	Function component
How it is created	A class that `extends Component`.	A plain function.
Where the UI is written	Inside the `render()` method, returned from it.	Returned directly from the function.
Inheritance / class syntax	Required (more complex).	Not needed (simple).
Weight	Heavier.	**Lightweight**; better performance (as explained by the instructor).
How it is displayed	Using its tag, e.g. `<Navbar />`.	Using its tag, e.g. `<NavigationBar />`.
Props	`this.props`	Function argument
State	Native feature (`this.state`, `this.setState`)	Needs a **hook** (`useState`)
Life cycle	Native predefined methods	Needs hooks

Which one should we use? (a student's question)

A student (Raju) asked how to decide between a function and a class. The instructor's answer: the same component can be written either way, and the output looks the same. If you want **lightweight components and better performance**, always go with the **function component** nowadays. Use a class only if you have a requirement to strictly follow object-oriented principles or an object-oriented design. Today most components are built as function components.

4.3 The single parent rule

Whether it is a class or a function, the component returns its UI inside `return ( ... )`. **Only a single parent element can be returned.** If you try to return multiple elements at the same level, React shows an error: *JSX expressions must have one parent element*.

Incorrect: error 'JSX expressions must have one parent element'

```
// WRONG - two elements at the same level
export function NavigationBar() {
  return (
    <div>
      <a href="#">Home</a>
    </div>
    <div>
      <img src="brand.png" />
    </div>
  );
}
```

Correct: wrapped in a single parent

```
// CORRECT - wrap everything inside one parent container
export function NavigationBar() {
  return (
    <div>
      <div>
        <a href="#">Home</a>
      </div>
      <div>
        <img src="brand.png" />
      </div>
    </div>
  );
}
```

The same error occurs when a `div` and an `img` are placed at the same level; wrap all elements that are at the same level inside one `div`. The instructor asked students to write the note: **JSX expressions must have one parent element.**

A student asked whether a Fragment can be used. The instructor said yes, an empty fragment can be used, and he would discuss Fragments later (not covered in this session).

Additional explanation (not from the lecture): A fragment is written as `<> ... </>`. It groups elements without adding an extra `div` to the page.

Part 5: Features of a Component - Props and State

A component, whether class or function, has two features: **props** and **state**. Every type of component supports both. We start with props.

Part 6: Props

6.1 What are props?

Props are nothing but the **attributes** (properties) of your component's tag. They are the attributes you pass in the tag. The word 'prop' is short for 'property'. In React we call attributes 'props'.

Consider predefined HTML tags. The `p` tag has the attribute `align` (value like right or justify). The `img` tag has `src`, `width`, `height` and `alt`. The anchor tag has `href`. In React these attributes are called props.

Predefined components already have predefined props (`src` for `img`, `href` for `a`). But who creates props for **your own** component? Only you, the developer. If you create your own component, you can create your own props for it, and pass them when you use the component.

6.2 Why do we need props?

Think of an HTML developer. The developer of the `img` tag does not know which image the user wants. The user supplies it with `src`. Similarly, the anchor tag developer does not know the link, so the user supplies `href`. In the same way, a component cannot decide all of its own data. **Props allow data to be passed dynamically into a component.** A component decides the structure of the UI, but the content (data) can come from outside.

The instructor's example: Home, About and Contact screens

Suppose you have three screens: **Home** (home.jsx), **About** (about.jsx) and **Contact** (contact.jsx). Each screen needs a section with the same layout: one heading (h1), one paragraph and one button.

Screen	Heading	Paragraph	Button
Home	Welcome to app	some text	button
About us	Welcome to about us	different text	button
Contact	View contact details	different text	button

The layout is the same everywhere, but the content differs. If you write the same 20 lines of HTML in every screen, you repeat code. The better way: if you can identify a piece of UI that repeats, **create a reusable component** for it. Here it is the **Intro** component (Intro.jsx).

While creating Intro, we know the structure (h1, p, button) but we **do not know the data**: the heading text, the paragraph content, and what the button should say. The Intro component cannot decide that data by itself. So the data is received through **props**.

Problem: first the static version (hard-coded)

Static Intro (limited)

```
// Intro with hard-coded text: the same text appears wherever it is used
export function Intro() {
  return (
    <div>
      <h1>Welcome to our app</h1>
      <p>This is our first React app</p>
      <button>Click me</button>
    </div>
  );
}
```

Every screen would show the same 'Welcome to our app'. That is why the instructor said the Intro component needs props (props are used when a component cannot decide the data itself).

Solution: pass the data through props

The developer defines props on the component. The instructor chose three props: **headingText**, **description** and **buttonText**. You can create any number of props, based on your requirement. They are passed as attributes in the

tag, like normal HTML attributes. The value is decided by the **parent** (the one who displays Intro).

Passing props

```
// src/App.jsx - the parent decides the values of the props
import { Intro } from './Intro';

function App() {
  return (
    <div>
      <Intro
        headingText="Welcome to our app"
        description="This is our first React app"
        buttonText="Click me"
      />
    </div>
  );
}

export default App;
```

6.3 How props reach the component (behind the scenes)

When you pass props, React creates **one object that contains all of the props** as key-value pairs (keys: headingText, description, buttonText). If the component is a **function component**, that object is passed as the **argument** of the function. The function must declare a parameter to receive it; the name `props` is recommended because it is readable. The argument of a function component always receives the props object only.

Intro receiving props

```
// src/Intro.jsx
export function Intro(props) {
  console.log(props);   // shows an object with all the props
  return (
    <div>
      <h1>{props.headingText}</h1>
      <p>{props.description}</p>
      <button>{props.buttonText}</button>
    </div>
  );
}
```

Expected output

```
Browser console:
{ headingText: "Welcome to our app",
  description: "This is our first React app",
  buttonText: "Click me" }

Page:
Welcome to our app
This is our first React app
[ Click me ]
```

Why the console shows an object: React first prepares an object of all the props passed to the component, then hands that same object to the function as its argument.

Displaying a variable inside HTML: curly braces

In plain HTML/JS, to show a JavaScript variable inside an element you would use DOM manipulation (`document.getElementById(...).innerHTML = value`). In React you do not have to manipulate the DOM yourself. It gives a shortcut: **open curly braces `{ }` in the JSX and write your JavaScript variable or property inside them.** Whatever the value is, it will be displayed there. So `{props.headingText}` shows the heading text inside the h1.

Reusing the same component with different data

Because the data comes from outside, you can render Intro again from another place with different values.

Rendering Intro twice with different props

```
function App() {
  return (
    <div>
      <Intro
        headingText="Welcome to our app"
        description="This is our first React app"
        buttonText="Click me"
      />
      <Intro
        headingText="About us"
        description="We have twenty years of experience in industry"
        buttonText="Know more about us"
      />
    </div>
  );
}
```

Expected output

```
Welcome to our app
This is our first React app
[ Click me ]

About us
We have twenty years of experience in industry
[ Know more about us ]
```

The structure is the same, but the data is different. The Intro component does not decide its data; the component that renders it (Home, About, Contact, Services, Gallery, Admissions ...) decides. This is the core requirement of props: **data transfer from one component to another**.

6.4 Destructuring the props

A student (Abhay) asked whether the props object can be destructured. The instructor said yes. Destructuring simply takes the properties out of the object into separate variables, so you do not have to write `props.` every time. React works exactly the same internally; it is only a coding style, and it makes the code more readable. It is better to destructure all the props first, then use them directly.

Destructuring inside the function

```
export function Intro(props) {
  const { headingText, description, buttonText } = props;
  return (
    <div>
      <h1>{headingText}</h1>
      <p>{description}</p>
      <button>{buttonText}</button>
    </div>
  );
}
```

Additional explanation (not from the lecture): You can also destructure directly in the parameter list: `export function Intro({ headingText, description, buttonText }) { ... }`. Note the curly braces in the parameter; they are required because the argument is an object.

6.5 Rules about props

The instructor asked: can a component change the value of its own props? Students suggested it is possible in JavaScript (like changing a function parameter). The instructor answered that technically you can, but ideally you should not. He gave the example of the `img` tag: the `src` value is data that the component should not modify. **Props are only there for read-only purposes.**

Rule	Meaning
Component cannot decide its own prop value	The value always comes from the parent component. (Default values are a different topic.)
Component should not change its props	Props are read-only. You can read them, make calculations, and use them to render the UI, but not change or initialise them.
Purpose	To pass data from one component to another (parent to child), e.g. App passes data to Intro.
Applies to	Both class and function components, in exactly the same way when passing.

Instructor's own summary: a component can neither decide the value of its own props nor change it. If you need data that a component can create and change itself, you should use **state** (Part 8).

Part 7: Props in a Class Component

Passing props is **exactly the same** for function and class components: you pass them as attributes in the tag. What differs is how the component **receives** them. A class has no function argument, so a different approach is used.

The instructor created **Introduction.jsx** as a class component with the same structure (h1, p, button).

Receiving props in a class: React again prepares an object of all props. But in a class, props become a **class member (property)**. The `Component` class already has a property named **props** that every subclass inherits (inheritance). Since it is a class-level property, it is accessed with the `this` keyword: `this.props`. It contains the entire props object.

Class component receiving props

```
// src/Introduction.jsx
import { Component } from 'react';

export class Introduction extends Component {
  render() {
    console.log(this.props);                        // whole props object
    const { headingText, description, buttonText } = this.props;   // destructure
    return (
      <div>
        <h1>{headingText}</h1>
        <p>{description}</p>
        <button>{buttonText}</button>
      </div>
    );
  }
}
```

Passing props to the class component (same syntax as before)

```
// src/App.jsx
import { Introduction } from './Introduction';

function App() {
  return (
    <div>
      <Introduction
        headingText="Know about us (class component)"
        description="This is a description for the class component"
        buttonText="Know more (class)"
      />
    </div>
  );
}
```

Expected output

```
Browser console:
{ headingText: "Know about us (class component)", description: "...", buttonText: "..." }
```

```
Page:
Know about us (class component)
This is a description for the class component
[ Know more (class) ]
```

A student (Om) asked about the constructor in this context. The instructor replied that for just receiving props you are not required to define a constructor.

Point	Function component	Class component
Passing props	`<Intro headingText="..." />`	`<Introduction headingText="..." />` (same)
Receiving props	As the **function argument**: `function Intro(props)`	As the **inherited class member**: `this.props`
Reading a value	`props.headingText`	`this.props.headingText`
Destructuring	`const { headingText } = props;`	`const { headingText } = this.props;`

Part 8: State in a Class Component

8.1 Why state? The limitation of props

So far: a component cannot decide its own prop value, and it cannot change it. But there are requirements where a component should **itself create** a piece of data, **decide its value**, and **change it later**. That feature is called **state**. The instructor summarised: **State is a type of property that a component can create by itself, decide its value, and also change its value.** A component is the complete owner of its state. State is available in class components and function components.

Point	Props	State
Who decides the initial value	The parent component.	The component itself.
Can the component change it?	No (read-only).	Yes, using the update mechanism (`setState`, or the setter from `useState`).
Purpose	Pass data from one component to another.	Data the component owns, creates and updates.
Effect on UI when changed	-	When state is updated, the component is **re-rendered**.

The scoreboard example

The instructor used a quiz scoreboard. It shows the current score (h1) and two buttons, plus (+) and minus (-). The score starts at **0**. A correct answer should increase the score by 3 (plus button). An incorrect answer decreases it by 1 (minus button). For example: start 0, first correct answer -> 3; second correct -> 6; third wrong -> 5. This score is data that the component itself must initialise (0) and update (+3 or -1). Such data is **state**; data that must be passed in from outside would be a prop.

> Why does state matter for which type of component? State is the **native feature of a class component**; it existed from day one of React. When function components were introduced, they lacked state. Later, state was made possible in functions through a workaround called **hooks** (Part 10). So first we study state in a class, because it is the native way.

8.2 Creating state with the constructor

State is a **class-level property named** `state` that the class inherits from `Component`. As with any class-level initialisation, **initialisation happens in the constructor**. The class has inheritance, so the constructor must call `super()` first (as learnt in object-oriented programming). Then assign the state with `this.state = ...`. The state can be an object, array, number, string, boolean, and so on; **an object is the ideal way**, because it can hold multiple states (score, player name, team data...).

```
constructor() {
  super();                      // mandatory call to the parent constructor
  this.state = { score: 0 };    // initialise state with an object
}
```

Because it is a class-level property, it is accessed with `this`. To **read** state in the UI use curly braces: `{this.state.score}`. Execution order: the **constructor runs first**; once it finishes, React calls `render()`.

Scoreboard stage 1 - displays the initial state

```
// src/Scoreboard.jsx  (stage 1: display only)
import { Component } from 'react';

export class Scoreboard extends Component {
  constructor() {
    super();
    this.state = { score: 0 };
  }

  render() {
    return (
      <div>
        <h1>Current score: {this.state.score}</h1>
        <button>+</button>
        <button>-</button>
      </div>
    );
  }
}
```

Expected output

```
Current score: 0
[ + ] [ - ]          (buttons do nothing yet)
```

Import `Scoreboard` in App.jsx and display `<Scoreboard />`. Nothing happens on click, because we have not handled the click events yet.

8.3 Handling events in React (onClick)

In plain HTML we use `onclick` and call a function manually. React's way is slightly different. The button has an event prop named **onClick**. The difference compared with HTML is the **naming convention**: in React, props follow **camelCase**. So `onclick` becomes `onClick`, and `onkeyup` becomes `onKeyUp` (a capital letter at the start of each new word).

The instructor first created two methods that just show an alert, to demonstrate event handling:

```
// Scoreboard stage 2 - events with alerts only
increaseScore = () => {
  alert('Score increased');
};

decreaseScore = () => {
  alert('Score decreased');
};

render() {
  return (
    <div>
      <h1>Current score: {this.state.score}</h1>
      <button onClick={this.increaseScore}>+</button>
      <button onClick={this.decreaseScore}>-</button>
    </div>
  );
}
```

Expected output

```
Click '+'  ->  alert: Score increased
Click '-'  ->  alert: Score decreased
```

Key rules

- Inside `onClick={ ... }` you open curly braces because you are writing JavaScript inside the HTML.
- You pass only the **reference** of the function: `this.increaseScore`. **Do not call it** (do not write the parentheses `this.increaseScore()`). React calls it internally when the user clicks.
- Because the functions are class-level members (properties), they must be accessed with `this.` in a class.

Common mistake: Writing `onClick={this.increaseScore()}`. This calls the function immediately while rendering instead of when the button is clicked.

Additional explanation (not from the lecture): The instructor wrote the handlers as class properties using the arrow-function form (`increaseScore = () => { ... }`). This form keeps `this` pointing at the component, which is needed to use `this.state` and `this.setState` inside the handler.

8.4 Updating state with setState

Now the real job: change the score. A student might think of writing `this.state.score = this.state.score + 3`. The instructor said you obviously can, but **you should not**; this approach is not appreciated. The correct way is to use another predefined function inherited from `Component`: `this.setState()`.

`setState` takes an object containing the **new state** (the keys and new values). To add 3, read the current score (`this.state.score`) and add 3. It does two jobs:

1. It **updates the state**.
2. It **re-renders the component** so that the UI shows the latest data.

Important: whenever state is updated, the component is re-rendered. If the state changed in memory but the UI was not re-rendered, the screen would keep showing the old data. That is why we use `setState`: it updates the state **and** triggers the re-render. This works the same for class and function components.

Complete Scoreboard class component

```
// src/Scoreboard.jsx  (final)
import { Component } from 'react';

export class Scoreboard extends Component {
  constructor() {
    super();
    this.state = { score: 0 };
  }

  increaseScore = () => {
    this.setState({ score: this.state.score + 3 });
  };

  decreaseScore = () => {
    this.setState({ score: this.state.score - 1 });
  };

  render() {
    return (
      <div>
        <h1>Current score: {this.state.score}</h1>
        <button onClick={this.increaseScore}>+</button>
        <button onClick={this.decreaseScore}>-</button>
      </div>
    );
  }
}
```

Execution flow (class component state)

1. The Scoreboard tag is displayed. React runs the **constructor**; `this.state = { score: 0 }`.
2. React calls **render()**. The h1 shows `0`.
3. The user clicks **+**. `increaseScore` runs: `this.state.score` is 0, so `setState({ score: 3 })`.

4. React **updates the state and re-renders** the component. `render()` runs again and now h1 shows `3`.
5. User clicks **+** again -> 6. User clicks **-** -> 5. Every update triggers a re-render.

Expected output

```
Initial:            Current score: 0
Click + :           Current score: 3
Click + :           Current score: 6
Click - :           Current score: 5
After page reload:  Current score: 0   (state starts again from the initial value)
```

While testing, the instructor got an error in the minus button because he had written `state.score` without `this.`; after changing it to `this.state.score` it worked. Always refer to state and class methods with `this`.

Concept	Class component syntax	Purpose
Create state	`this.state = { ... }` inside the constructor	Initial value of the component's data
Read state	`this.state.score`	Display or use the current value
Update state	`this.setState({ score: ... })`	Change the value **and** re-render
Handle click	`onClick={this.increaseScore}`	Run the function when the user clicks (pass reference only)

Common mistake: Updating the state by assigning directly (`this.state.score = 10`). The instructor said not to do this; use `setState` so the component is re-rendered with the new value.

Part 9: Virtual DOM

A student asked which component re-renders when state changes (only the scoreboard, or the entire screen?). To answer, the instructor introduced the concept of the **Virtual DOM**, the mechanism working behind the scenes in React.

9.1 The DOM and the virtual DOM

DOM (Document Object Model): when a web page loads in the browser, the browser prepares a DOM **tree structure** by reading the page. Then it renders the whole web page from that tree.

React creates a **virtual copy of the actual browser DOM**. This copy is the **virtual DOM**. Operations happen on the virtual DOM first, and then the virtual DOM is translated into the actual DOM, and the browser displays the result.

9.2 The apparent contradiction

The instructor posed a question: React says 'the virtual DOM is faster than the actual DOM', but the virtual DOM is just a copy, and it is translated into the actual DOM, so we added one **extra layer / extra step**. How can this be faster?

9.3 Why it is faster

In plain HTML, CSS and JavaScript, if you update even one small thing (change an h1 or a paragraph after a button click), the browser recreates **the entire DOM tree structure**, even though you changed only a single element out of, say, a thousand tags. That re-creation and re-rendering of the whole tree is a slow, time-consuming process. This happens when you do DOM manipulation by hand.

With React, whenever the UI is going to change, React:

1. Builds the **new** virtual DOM.
2. **Compares** it with the virtual DOM prepared the last time (the previous copy).
3. Finds **only the difference** (for example, only the h1 has new data).

4. Updates **only that part** of the real DOM. The rest of the page is not re-rendered.

Example: the score changes from 2 to 5. The button and other elements are the same. Only the h1 element has new data. So only that h1 is updated. The time saved by updating only the changed section is much larger than the small time spent on the extra comparison and translation. That is why the virtual DOM is faster than the actual DOM approach. The instructor warned this is a typical **interview question**: 'Why is the virtual DOM faster than the actual DOM?'

> A student (Abhay) asked whether the whole component re-renders or only the changed element. The instructor said the component function runs again (re-renders), but only the differing part updates in the real DOM. To avoid unnecessary re-rendering of components, you can later use memoization with `useMemo` and `useCallback` (to be learned later).

Point	Actual DOM (manual DOM manipulation)	Virtual DOM (React)
What is it	The real tree the browser builds.	A copy of the real DOM maintained by React.
On a small change	Recreates/re-renders the entire tree.	Compares old and new virtual DOM and updates **only the difference**.
Speed	Slow for large pages.	Faster overall, despite the extra translation step.

Part 10: Hooks and State in a Function Component

Now the same state behaviour has to be implemented in a **function component**. The instructor repeated that state is **not** a native feature of a function component. A function does not have a `this.state` property or a `setState` function.

10.1 What is a hook?

Not only state: other features, such as the **life cycle**, are native to class components only. To get these class features in a function component, React introduced a concept called **hooks**. The analogy: in real life, a hook is used to hang an external thing. State is an 'external behaviour', a class component feature, and with hooks we **hang** that behaviour on our function component. So the concept is called a hook.

What is a hook technically? **A hook is just a regular JavaScript function** (a special, predefined function). It hooks a class-native feature into your function component. For example, one hook function gives you state in a function component.

Naming convention

Every hook function name should **start with the prefix** `use`. This is a recommendation we should always follow: `useState`, `useMemo`, `useCallback`, `useParams`, `useRouter`, `useNavigate`. Whenever a function has the prefix `use`, it is a hook. If you write your own hook, follow the same naming convention.

10.2 useState

The hook to create a state in a function component is `useState`. It is imported from the React package.

Part	Explanation
`useState(initialValue)`	Call it and pass the **initial value** of the state (for score: `0`).
Return value	It returns an **array of two things**: (1) the current state value, (2) a function that updates this state.
Array destructuring	Because it returns an array, we destructure it: `const [score, setScore] = useState(0);`
Naming	First element: a readable name like `score`. Second: `set` + name, like `setScore`.

Why destructure? From arrays to readable names

If you store the result in a variable `rs`, then `rs[0]` is the state value and `rs[1]` is the update function. That is not readable. Array destructuring lets you take the two elements out directly and give them proper names. The names `x` and `y` would also work, but they are not readable names, so always choose meaningful names.

```
// Not readable
const rs = useState(0);
// rs[0] -> state value, rs[1] -> function to update it

// Recommended (array destructuring)
const [score, setScore] = useState(0);
```

10.3 Scorecard function component

Complete Scorecard function component

```
// src/Scorecard.jsx
import { useState } from 'react';

export function Scorecard() {
  const [score, setScore] = useState(0);     // state: score, initial value 0

  const increaseScore = () => {
    setScore(score + 3);                     // read current score, add 3
  };

  const decreaseScore = () => {
    setScore(score - 1);                     // read current score, subtract 1
  };

  return (
    <div>
      <h1>Score: {score}</h1>
      <button onClick={increaseScore}>+</button>
      <button onClick={decreaseScore}>-</button>
    </div>
  );
}
```

Explanation

- `useState(0)` creates the state with initial value 0 and returns the array `[value, updaterFunction]`.
- `score` is read directly (no `this.`); it is just a variable in the function.
- `increaseScore` and `decreaseScore` are plain functions defined in the component. Since it is a function component, they are referenced directly (**no** `this.`).
- `onClick={increaseScore}` passes the reference only (do not call it).
- `setScore(score + 3)`: the argument is the **new value** for that state. When the state is updated, the component is **re-rendered**, as with class components.

Expected output

```
Initial:    Score: 0
Click + :   Score: 3
Click + :   Score: 6
Click - :   Score: 5
```

Display `<Scorecard />` in App.jsx. The Scoreboard (class) and the Scorecard (function) behave in the same way.

10.4 Multiple states in a function component

When you need more than one state, call `useState` once for each. Each state gets its own value and its **own** update function (unlike a class, which has only one `setState` for all).

```
const [score, setScore] = useState(0);        // number
const [player, setPlayer] = useState('');     // text, initially blank
const [users, setUsers] = useState([]);       // array, initially empty
```

```
// to update: setScore(...)   setPlayer(...)   setUsers(...)
```

A student (Abhay) asked whether the state in a function component must be a single object like in a class (`this.state = { score: 0 }`) or can be a number directly. The instructor explained: in a function component you can use **standalone, discrete values** (a number, string, array) with one `useState` call per state. In a class, `this.state` is a single object holding different properties.

Point	Class component	Function component
Create state	`this.state = { score: 0 }` in the constructor	`const [score, setScore] = useState(0)`
Read state	`this.state.score`	`score`
Update state	`this.setState({ score: ... })` (one function for all)	`setScore(...)` (one function **per state**)
Structure	One object for all state	Separate `useState` per value
Native?	Yes, native feature	No, achieved through the `useState` hook

Part 11: Component Life Cycle

11.1 What is a life cycle?

The instructor asked 'What is a life cycle?' Students answered: 'You create a component, use it, and then destroy it' and 'the duration from start to end'. The instructor agreed: **the life cycle is the duration from the start to the end** of something. A person is born, goes to school, college, job; that is a life cycle. Similarly, in every programming domain, an entity is created, operations are performed, and it ends, in a particular sequence. The class had already learnt the life cycle of a **thread in Java**, where predefined methods (start, sleep, etc.) are called at different stages.

A React component also has a life cycle (class or function). It controls the start, the operations in between, and the end of the component. React provides **predefined functions** that are **called automatically at each life cycle stage**. With these you can decide what happens at the start, at the end, and in between. These life cycle functions are **natively available in class components** (defined in the `Component` class and inherited). Function components need **hooks** to get the same features.

11.2 Mounting and unmounting

When a component is displayed on the screen, we say it is **mounted**. 'Mount' means to attach: React attaches the component to the web page so that it appears. When a component is removed from the screen (for example, when a route changes from Home to About Us and the Home component disappears), that is **unmounting**.

Term	Meaning	Functions in the lecture
Mounting phase	The component is displayed (attached) on the screen.	constructor, render, componentDidMount
Update	State changes while the component is shown.	render runs again
Unmounting phase	The component is removed from the screen.	(introduced as a concept; no function was demonstrated)

11.3 The three functions of the mounting phase

1. constructor

The constructor is called **only once**, when the component is mounted. It does the initialisation job, such as creating state and properties. It is called a single time in the whole life cycle.

2. render

When the constructor finishes its job, **render** is called, and it provides the UI for the component. The instructor asked: while the constructor runs, will render start immediately? **No, render waits for the constructor.** Until render runs, nothing is shown on the screen, not even a loader, because render has not provided any UI. The user sees a blank white screen.

Render can run many times. The first time, it runs after the constructor. After that, if any **state is updated**, React calls render again. So the constructor runs once, but render runs again and again, depending on state updates.

3. componentDidMount

This function is also a part of the life cycle. It is called **just after the first render call**. The instructor explained the name as 'component did mount': the component has mounted. It is called **only once**, immediately after the first render. If render runs again later because of a state update, **componentDidMount is not called again**.

Function	When called	How many times	Typical use
`constructor()`	First, when the component is mounted	Once	Initialise state and properties (call `super()` first)
`render()`	After the constructor; again after every state update	Many times	Return the UI only
`componentDidMount()`	Just after the first render	Once	Work to do after the first render, such as fetching data from a server

Proving the order with console.log

Logging the life cycle

```
// src/MyComponent.jsx (and also tried inside the Scoreboard)
import { Component } from 'react';

export class MyComponent extends Component {
  constructor() {
    super();
    console.log('Constructor called');
    this.state = { score: 0 };
  }

  componentDidMount() {
    console.log('componentDidMount called');
  }

  increaseScore = () => {
    this.setState({ score: this.state.score + 3 });
  };

  render() {
    console.log('Render called');
    return (
      <div>
        <h1>Score: {this.state.score}</h1>
        <button onClick={this.increaseScore}>+</button>
      </div>
    );
  }
}
```

Expected output

```
On first load (console):      Constructor called
                              Render called
                              componentDidMount called

After clicking + (state update):   Render called
                                    (componentDidMount is NOT called again)
```

In the lecture the instructor saw things running twice in the console. The reason: the project is running in development mode with **Strict Mode** enabled (main.jsx), so React runs some functions twice to help find bugs. The order is still constructor, render, then componentDidMount (after the first render). Updating the state calls only render again and again, not componentDidMount.

Additional explanation (not from the lecture): Strict Mode's extra execution happens only in development, not in a production build.

Part 12: Fetching Data with componentDidMount

12.1 What is componentDidMount used for?

A student (Abhay) asked what componentDidMount is used for. The instructor answered with a real-life example: when you open Instagram, YouTube, Facebook or WhatsApp Web, you do not click a button 'show me the feed'. The posts, videos, timeline or chats appear as soon as the application loads. So the data is fetched **as soon as the component loads**. (The events for fetching can be different: on a button click, while the user types, or at component load.) For the 'at load' requirement, where do we fetch?

Where	Problem if we fetch here
Constructor	Render waits for the constructor to finish. If the API takes 10 to 20 seconds, the user sees a **blank screen** for that long; we cannot even show a loader, because render has not run.
Render	Render has only one job: **display the UI**. Also the first render does not yet have any server data.
componentDidMount	**Correct place.** It runs after the first render, so the user already sees the static UI (navbar, footer, loader or skeleton) while data is being fetched.

The instructor's recommended approach: in the **first render**, show whatever is static (the navbar, footer) and a **loader**, a **message** like 'Please wait, we are fetching the data', or a **skeleton loader / shimmer effect** (the grey placeholder boxes you see on apps while content loads). As soon as the first render completes, componentDidMount runs and fetches the data from the API. When the data arrives, **assign it to the component state**; updating state triggers render again, and this time the data is available.

Instructor's summary: we use this life cycle function to fetch data at the time of component load. It can perform other tasks needed just after the first render too, but data fetching is the most popular use. Hold API data in state: create a state, initialise it, and when data arrives assign it to the state.

12.2 Building the UsersList component

The requirement: fetch the users' data from a fake API (**JSONPlaceholder**, which the class had used earlier) and display it in a table.

Stage 1: fetch and log on the console

Create **UsersList.jsx** as a class component, with a constructor (calling `super()`), `componentDidMount` and `render`. In componentDidMount use the JavaScript `fetch` function.

- `fetch(url)` sends a **GET** request by default and returns a **Promise**.
- Use `await` to get the response; therefore mark the function `async`.
- `response.json()` also returns a promise; `await` it to get the actual users data.
- For error handling, the instructor simply logged the error to the console.

UsersList stage 1

```
// src/UsersList.jsx  (stage 1)
import { Component } from 'react';

export class UsersList extends Component {
  constructor() {
    super();
```

```
    console.log('Inside constructor of UsersList');
  }

  async componentDidMount() {
    console.log('Inside componentDidMount of UsersList');
    try {
      const response = await fetch('https://jsonplaceholder.typicode.com/users');
      const usersData = await response.json();
      console.log(usersData);
    } catch (error) {
      console.log(error);
    }
  }

  render() {
    console.log('Inside render of UsersList');
    return (
      <div>
        <h4>List of users</h4>
      </div>
    );
  }
}
```

Additional explanation (not from the lecture): The URL `https://jsonplaceholder.typicode.com/users` is the standard JSONPlaceholder users endpoint; the lecture said only 'JSON placeholder'. The `try/catch` is the usual way to catch errors in `async/await` code.

In App.jsx comment out the earlier components (Intro, Introduction, Footer, imports) so they remain as reference, then import and render `<UsersList />`.

Expected output

```
Page:    List of users
Console (in order):
  Inside constructor of UsersList
  Inside render of UsersList
  Inside componentDidMount of UsersList
  [ { id: 1, name: "...", username: "...", email: "...", phone: "...", ... }, ... ]
```

This proves the order: constructor -> render -> componentDidMount -> data fetched -> data logged. But we must show the data on the **UI**, not only on the console.

Stage 2: table structure

A table was added with a header section (`thead`) and a body section (`tbody`). The header row has five `th` cells: **ID, Name, Username, Phone, Email**. Table attributes were passed as props with camelCase names: `border="1"`, `cellPadding="10"` and `cellSpacing="0"`.

Stage 3: why the fetched data cannot be used directly in render

The data arrives in componentDidMount in a **local variable** (`const usersData`). A `const` is accessible only within its block, so **render cannot use it directly**. Data is fetched by componentDidMount but needed by render. The solution: **use state**. The instructor said that you will do this every time you need to hold API data in a component (users, products, customers, whatever API you call): create a state, initialise it, and assign the data into it when it arrives. It does not matter whether you fetched on componentDidMount, a button click, or while the user types.

```
constructor() {
  super();
  this.state = { users: [] };       // empty array: we do not yet know the data
}

async componentDidMount() {
  const response = await fetch('https://jsonplaceholder.typicode.com/users');
  const usersData = await response.json();
  this.setState({ users: usersData });   // data arrives -> update state -> re-render
}
```

Initially the state holds an empty array. When the data arrives, `setState` replaces it with the actual array, and because state changed, **render is called again**, and this time render can read `this.state.users`. Render asks the **state** for the data (not componentDidMount).

12.3 Rendering a list in React (list rendering)

Now we must show an array as UI elements, i.e. create multiple table rows dynamically. In plain HTML we used DOM manipulation: `createElement('tr')`, loop through the array, create `td`s, set `innerHTML`, append... 'a lot of mess', as the instructor put it. In React you do **not** have to do that.

The React idea

React says: do not manually create the elements. **Give me an array of the UI elements (components/tags) you want to display, and I will render all of them.** So the job is: read the data array and create a **new array of HTML elements/components** from it. The way to create a new array by reading an existing array is the `map()` function (learnt earlier). map runs a callback for each element and returns a new array of whatever the callback returns.

Example 1: array of numbers -> paragraphs

Suppose the state is `myList: [10, 20, 30, 40, 50]` and we want each number in its own paragraph. Decide the UI first: **one paragraph per element**. Then use map to create an array of paragraphs.

map() creates an array of <p> elements

```
constructor() {
  super();
  this.state = { myList: [10, 20, 30, 40, 50] };
}

render() {
  return (
    <div>
      {this.state.myList.map((value) => (
        <p>{value}</p>
      ))}
    </div>
  );
}
```

Expected output

```
10
20
30
40
50
```

How it works: `this.state.myList.map(...)` runs the callback 5 times. The callback receives each number as `value` and returns a paragraph containing it. The result is `[<p>10</p>, <p>20</p>, <p>30</p>, <p>40</p>, <p>50</p>]`. React is smart enough to display all elements of this array. The curly braces `{ }` around map are needed because we write JavaScript inside the HTML.

Example 2: array of objects -> div with h4 and p

Now the state is an array of objects, each with `id` and `name`:

```
this.state = {
  myList: [
    { id: 111, name: 'Arun' },
    { id: 112, name: 'Aman' },
    { id: 113, name: 'Suraj' },
  ],
};
```

Required UI for each object: one `div` containing an `h4` (the id) and a `p` (the name). We need an array of such divs.

map() with objects

```
render() {
  return (
    <div>
      {this.state.myList.map((value) => (
        <div>
          <h4>{value.id}</h4>
          <p>{value.name}</p>
        </div>
      ))}
    </div>
  );
}
```

Expected output

```
111
Arun
112
Aman
113
Suraj
```

In each iteration, `value` holds a single object (first iteration: the first object, second iteration: the second, and so on). `value.id` goes into the h4 and `value.name` into the p. The instructor's analogy: 'You have built a machine; you put the potato in one side and gold comes out the other': you put the data array in, and an array of UI elements comes out. React then reads that array itself and renders the elements.

Key idea of list rendering: **do not do DOM manipulation or write loops to create elements manually.** Read the data array, create an array of your UI elements using map, and give it to React. React renders them.

Additional explanation (not from the lecture): React normally expects a unique `key` prop on each element in a list, for example `<div key={value.id}>`. The instructor did not discuss `key` in this part of the lecture. Without it, React shows a warning in the console, although the page still renders.

12.4 Rendering the users table rows

Apply the same technique to the table body. We need an array of `tr` elements, each containing five `td`s: id, name, username, phone and email. We have the data array in `this.state.users`.

Rows created with map

```
<tbody>
  {this.state.users.map((user) => (
    <tr>
      <td>{user.id}</td>
      <td>{user.name}</td>
      <td>{user.username}</td>
      <td>{user.phone}</td>
      <td>{user.email}</td>
    </tr>
  ))}
</tbody>
```

Execution flow (a dry run, as done in class)

1. The **constructor** runs first. It initialises `this.state = { users: [] }`: an empty array, because there is no data yet.
2. **render** runs (first render). `map()` on an **empty array** runs zero times, so the `tbody` is empty. The user sees a table with only the header row. The instructor stressed: **in the first render, the table never shows the data.**
3. **componentDidMount** runs just after the first render. It fetches the data from the server and calls `setState`, storing the data in state.
4. Since the state is updated, **render runs again** (second render). This time `this.state.users` has the data, so map creates all the tr elements.
5. The table now shows the data: **the second render** shows the data.

Expected output

```
First render:   header row only (ID | Name | Username | Phone | Email), empty body
After ~1 s:     header row + one row per user (the data appears on the second render)
```

Proving it with a delay (setTimeout)

To make the two renders visible, the instructor wrapped the fetch code inside `setTimeout` with a delay of 3 seconds (3000 ms) and put `async` on the callback function. After reloading, the user first saw an empty table and after three seconds, the data appeared.

```
componentDidMount() {
  setTimeout(async () => {
    const response = await fetch('https://jsonplaceholder.typicode.com/users');
    const usersData = await response.json();
    this.setState({ users: usersData });
  }, 3000);                      // 3 seconds delay, only to demonstrate the two renders
}
```

12.5 Conditional rendering (showing a loading message)

Showing an empty table for 3 seconds is poor for the user. Better: while the data is empty, show a message such as 'Loading, please wait' (or 'No user found'); once the data arrives, show the real data. Deciding **what to render based on a condition** is called **conditional rendering**: if the condition is true, one thing is rendered (component C1), and if false, another (component C2). What we did before (map) was **list rendering**; this is **conditional rendering**.

It is done with the ordinary **conditional (ternary) operator**: `condition ? valueIfTrue : valueIfFalse`. Condition: the length of `this.state.users`. If it is **0**, show the loading message inside the `tbody`; otherwise, run the map code to show the real rows.

Conditional rendering with the ternary operator

```
<tbody>
  {this.state.users.length === 0 ? (
    <tr>
      <td colSpan="5">Loading...</td>
    </tr>
  ) : (
    this.state.users.map((user) => (
      <tr>
        <td>{user.id}</td>
        <td>{user.name}</td>
        <td>{user.username}</td>
        <td>{user.phone}</td>
        <td>{user.email}</td>
      </tr>
    ))
  )}
</tbody>
```

Additional explanation (not from the lecture): The instructor showed the message directly inside the tbody. Wrapping it in `tr` and `td` (with `colSpan="5"`) makes valid table HTML.

Expected output

```
First 3 seconds (delayed demo):    Loading...
After the data arrives:            ID | Name | Username | Phone | Email
                                   1  | ...  | ...      | ...   | ...
                                   (one row for every user)
```

Complete final UsersList component

```
// src/UsersList.jsx  (final version)
import { Component } from 'react';

export class UsersList extends Component {
  constructor() {
```

```
    super();
    console.log('Inside constructor of UsersList');
    this.state = { users: [] };
  }

  async componentDidMount() {
    console.log('Inside componentDidMount of UsersList');
    try {
      const response = await fetch('https://jsonplaceholder.typicode.com/users');
      const usersData = await response.json();
      this.setState({ users: usersData });
    } catch (error) {
      console.log(error);
    }
  }

  render() {
    console.log('Inside render of UsersList');
    return (
      <div>
        <h4>List of users</h4>
        <table border="1" cellPadding="10" cellSpacing="0">
          <thead>
            <tr>
              <th>ID</th>
              <th>Name</th>
              <th>Username</th>
              <th>Phone</th>
              <th>Email</th>
            </tr>
          </thead>
          <tbody>
            {this.state.users.length === 0 ? (
              <tr>
                <td colSpan="5">Loading...</td>
              </tr>
            ) : (
              this.state.users.map((user) => (
                <tr>
                  <td>{user.id}</td>
                  <td>{user.name}</td>
                  <td>{user.username}</td>
                  <td>{user.phone}</td>
                  <td>{user.email}</td>
                </tr>
              ))
            )}
          </tbody>
        </table>
      </div>
    );
  }
}
```

- A student asked: 'You did not use a for loop here?' The instructor replied: yes, the `map` function just builds the array; React renders all those elements automatically. **You do not manually run for loops or do DOM manipulation.** That is the ease React gives.
- Common pattern: create state -> initialise it (empty array) -> fetch data (componentDidMount, button click, typing, or any event) -> assign data to state -> render reads state.

Concept	What it means	Example from the lecture
List rendering	Convert a data array into an array of UI elements using `map()`	`users.map(user => <tr>...</tr>)`
Conditional rendering	Render different UI based on a condition (ternary operator)	`users.length === 0 ? Loading : rows`

Part 13: Class Wrap-up - MCP Discussion and Human Verification

After the main topic, a student asked about **MCP** (Model Context Protocol), asking whether it is similar to an API. This was a side discussion, not part of the React syllabus. The instructor's explanation:

- MCP is like an API in the sense that it **integrates an AI agent with a third-party application**, so that further things can be automated.
- **Example 1 (daily summary mail):** a manager asks the team to mail a summary every day (what was done, what is pending). The developer used Claude to write the code, so Claude already knows what is complete and pending. With an MCP, you create a connection to Outlook and tell it: every day at 6 PM, fetch the summary from Claude and post a mail to the manager. Claude and Outlook are linked through the MCP server. The instructor said he built this use case in practice.
- **Example 2 (timesheets):** in companies, developers fill in a timesheet of the work done each day, and this is used to bill customers. It is a hectic process, often left pending until month end. An MCP can fetch the summary from Claude and fill the timesheet portal date-wise.
- **Example 3 (PR review):** pull requests on Git can be reviewed by asking Claude through an MCP.
- MCP can integrate multiple applications to automate such tasks.

> Closing remark from the instructor: **human intervention must always be present to verify the data** produced by AI in such cases. This is why software engineers' jobs still exist.

The instructor also said he would share the previous day's notes and code files (he had forgotten to share them), and the session then ended. A break/lab instruction (lunch, then rejoin the lab using the same link, with breakout rooms) was announced by another staff member at the end.

Part 14: Revision Material

14.1 Concise summary of all topics

1. **Setup:** `npm create vite@latest`, choose React + JavaScript; `npm i`; `npm run dev` (port 5173). Clean App.jsx and remove unused CSS imports.
2. **Class component:** `class X extends Component`, override `render()`, return UI, `export`, import elsewhere, display with `<X />`.
3. **Tags:** every component automatically gets a tag with its own name. Every tag, predefined or custom, must be closed (`<br />`).
4. **JSX:** JavaScript + XML; write HTML inside JS in `.jsx` files; XML origin explains the closing-tag rule.
5. **Function component:** a function that returns UI directly; lightweight; used the same way as a class component.
6. **Single parent rule:** a component must return one parent element ('JSX expressions must have one parent element').
7. **Props:** attributes passed to a component; read-only; parent decides value; function component receives them as argument, class receives them as `this.props`; `{ }` displays JS values; destructuring is optional but cleaner.
8. **State:** data the component owns, decides and changes. Class: `this.state` in constructor + `this.setState()`. Updating state re-renders the component.
9. **Events:** `onClick={this.fn}` in camelCase; pass function reference, never call it.
10. **Virtual DOM:** React's copy of the DOM; compares old and new copies and updates only the difference, which is why it is faster.
11. **Hooks:** regular functions (prefix `use`) that give class features to function components. `useState(initial)` returns `[value, setter]`.

12. **Life cycle:** mounting (constructor -> render -> componentDidMount) and unmounting. Constructor once; render many; componentDidMount once after first render.
13. **Fetching data:** use componentDidMount, store the result in state, render reads state; show a loader/skeleton in the first render.
14. **List rendering:** `array.map()` returns an array of elements; React renders it. **Conditional rendering:** ternary operator.

14.2 Important definitions and syntax

Term	Definition	Syntax / example
Component	A reusable piece of UI, either a class or a function.	`export function NavigationBar() { return (...); }`
Class component	A class that extends `Component` and has `render()`.	`class X extends Component { render() { return (...); } }`
Function component	A function that returns UI.	`function X(props) { return (...); }`
JSX	JavaScript and XML; HTML inside JS.	`.jsx` files
Props	Read-only attributes passed to a component.	`<Intro headingText="Hi" />`
State	Data owned and changed by the component.	`this.state = { score: 0 }`
setState	Updates state and re-renders (class).	`this.setState({ score: 3 })`
Hook	A function that adds class features to a function component. Name starts with `use`.	`const [score, setScore] = useState(0)`
Virtual DOM	React's copy of the DOM used to update only the differences.	-
Mounting	Component being displayed on screen.	constructor, render, componentDidMount
Unmounting	Component being removed from the screen.	-
componentDidMount	Runs once, just after the first render.	`componentDidMount() { ... }`
List rendering	Showing an array as multiple UI elements using map.	`arr.map(x => <p>{x}</p>)`
Conditional rendering	Rendering different UI by a condition.	`cond ? <A /> : <B />`

14.3 Key differences between related concepts

Comparison	Difference
Props vs State	Props are passed from outside and are read-only. State is created, owned and changed by the component itself.
Class vs Function component	Class: extends Component, render(), native state and life cycle. Function: plain function, lightweight, uses hooks for state/life cycle.
this.props vs props argument	`this.props` in a class component; `props` function argument in a function component.
this.setState vs useState setter	Class has one setState for all state; a function has a separate setter for each useState.
constructor vs componentDidMount	Constructor: initialisation, before first render. componentDidMount: after first render, ideal for fetching data.
HTML vs XML	HTML displays data on the page with predefined tags. XML stores data with user-defined tags.
Actual DOM vs Virtual DOM	Actual DOM re-renders the whole tree on change. Virtual DOM compares and updates only what changed.
List vs Conditional rendering	List: map an array to many elements. Conditional: choose what to show with a condition.
onClick={fn} vs onClick={fn()}	First passes a reference (runs on click). Second calls the function immediately while rendering.

14.4 Common mistakes to avoid

Mistake	Correction
Class not extending Component	Use `extends Component`; otherwise it is a normal class.
Forgetting `render()` or `return` in a class	Define `render()` and return the JSX.
Forgetting export/import	`export` the component and `import` it in the file where it is displayed.
Using an unclosed tag, e.g. `<input>`	Close every tag: `<input />`.
Returning two sibling elements	Wrap them in one parent (`div` or fragment).
Writing `onclick` in lowercase	Use camelCase: `onClick`.
Calling the handler: `onClick={this.fn()}`	Pass reference: `onClick={this.fn}`.
Forgetting `this.` in a class	Use `this.state`, `this.props`, `this.fn`.
Using `this.` in a function component	Not needed; use variables directly.
Forgetting `super()` in the constructor	Call `super()` before using `this`.
Assigning state directly	Use `setState` (class) or the setter (function).
Trying to change props	Props are read-only.
Fetching data in constructor/render	Fetch in componentDidMount and keep results in state.
Using a local variable from componentDidMount in render	Store the data in state, then read state in render.
Expecting data in the first render	The first render has no data; handle empty data (loader/conditional rendering).

14.5 Important interview questions and answers

Questions marked (L) were pointed out or discussed in the lecture. Others are extra practice (beyond the lecture) based on the same topics.

#	Question	Answer
1 (L)	What is JSX?	JavaScript and XML. An extension that allows writing HTML directly inside JavaScript code.
2 (L)	Why is the virtual DOM faster than the actual DOM?	React compares the new virtual DOM with the previous copy and updates only the changed part of the actual DOM instead of recreating the whole tree.
3 (L)	What are the two ways of creating a component?	Class component and function component.
4 (L)	How do you convert a normal class into a React component?	Extend the `Component` class and define the `render` method.
5 (L)	What is the role of `render()`?	It returns the UI of a class component; React calls it to display the component.
6 (L)	What are props?	Attributes passed to a component. They carry data from parent to child and are read-only.
7 (L)	Can a component change its props?	No, props are read-only; a component also cannot decide their value.
8 (L)	What is state?	Data a component creates, decides and changes itself. Updating it re-renders the component.
9 (L)	How do you update state in a class component?	With `this.setState()`, never by direct assignment.
10 (L)	What does `setState` do apart from updating state?	It triggers a re-render of the component.
11 (L)	What is a hook?	A regular function (name starts with `use`) that brings class features such as state and life cycle into function components.
12 (L)	What does `useState` return?	An array: [current state value, function to update it].

#	Question	Answer
13 (L)	Why can't we use a state in a function directly like in a class?	State is a native feature of class components; function components need the `useState` hook.
14 (L)	What is the life cycle of a component?	The duration from its start (mounting) to its end (unmounting), with predefined functions called at each stage.
15 (L)	How many times are constructor, render and componentDidMount called?	Constructor once, componentDidMount once (after first render), render many times (after every state update).
16 (L)	Where do you fetch data when a component loads, and why?	In componentDidMount; the constructor would delay render (blank screen), and render is only for UI.
17 (L)	How do you render a list in React?	Use `map()` on the array to create an array of elements and place it inside curly braces in JSX.
18 (L)	What is conditional rendering?	Rendering different UI depending on a condition, e.g. the ternary operator.
19	Why must JSX return a single parent element?	A function can return only one value; wrap siblings in one parent (or a fragment).
20	Why use `key` in lists?	Beyond the lecture: helps React identify which items changed. Use a unique id, e.g. `key={user.id}`.
21	Difference between `onClick={fn}` and `onClick={fn()}`?	First runs on click; the second runs immediately during render.

14.6 Practice exercises

Solutions are intentionally not given.

Easy

1. Create a new Vite React project and clean App.jsx so that it shows only an h1. Remove the unused CSS imports.
2. Create a class component `Header` with a title and two links, and display it in App.
3. Create a function component `Footer` that returns a paragraph. Display both Header and Footer in App.
4. Write one wrong example that returns two sibling elements and fix it.
5. Change `<input>` to a valid React tag and explain why.

Medium

1. Create a reusable `Card` function component with props `title`, `text` and `buttonText`, and render it three times with different data.
2. Create the same Card as a class component using `this.props`, and destructure the props.
3. Create a class component `Counter` with state `count`. Add buttons to increase by 2 and decrease by 1.
4. Create the same `Counter` as a function component using `useState`.
5. Add `console.log` statements to constructor, render and componentDidMount, click a button, and write down the order you observe and why.

Challenging

1. Add a second state `player` (text) to the function-component counter and a button that changes it.
2. Create a class component with state `products` (array of objects with `id`, `name` and `price`) and display them as a list of `div`s using `map`.
3. Build a `PostsList` class component that fetches posts from JSONPlaceholder in componentDidMount, stores them in state, and shows them in a table.
4. Add a 'Loading...' message using conditional rendering and test it by adding a 3-second `setTimeout`.
5. Explain in your own words, with a diagram, what happens in each render of PostsList (first render, componentDidMount, second render).

14.7 Final revision checklist

[ ] I can create a React app with Vite and run it with npm run dev.

[ ] I can write a class component (extends Component, render, export, import, tag).

[ ] I know that every tag in React must be closed, and why (XML).

[ ] I can explain what JSX is and the difference between HTML and XML.

[ ] I can write a function component and use it exactly like a class component.

[ ] I know the single parent rule for JSX.

[ ] I can explain props and the read-only rule, and pass/receive props in function and class components.

[ ] I can use curly braces to display JavaScript values and can destructure props.

[ ] I can explain state and create/update it in a class (constructor, this.state, setState).

[ ] I can handle onClick correctly (camelCase, pass reference, not call).

[ ] I can explain the virtual DOM and why it is faster.

[ ] I can explain hooks, the `use` naming rule, and use `useState` with array destructuring.

[ ] I know mounting, unmounting, and the order and frequency of constructor, render, componentDidMount.

[ ] I know why data is fetched in componentDidMount (not constructor or render) and stored in state.

[ ] I can render a list with map and use conditional rendering for a loading message.

[ ] I can explain what MCP is in simple words (from the side discussion).

14.8 Completeness check against the transcript

Each topic discussed in the lecture, in order, with the part of these notes where it is covered:

Topic in the lecture	Covered in
Creating Vite React app, npm i, npm run dev, port 5173, VS Code	Part 1.1 - 1.2
Cleaning App.jsx, removing CSS and imports	Part 1.3
Navbar class component: extends Component, render, return, export, import, tag	Part 2.1 - 2.2
Behind-the-scenes render call; every component gets a tag	Part 2.2
Footer class component	Part 2.3
Closing tags and self-closing rule	Part 2.4
JSX, XML vs HTML, interview question	Part 3
Function component (NavigationBar), class vs function choice	Part 4.1 - 4.2
Single parent element rule; Fragment mention	Part 4.3
Features of a component: props and state	Part 5
Props: attributes, Intro reusable component, props object, curly braces, destructuring, read-only rule	Part 6
Props in class component (this.props)	Part 7
State: need, scoreboard, constructor, events (onClick), setState, re-render	Part 8
Virtual DOM and why it is faster; useMemo/useCallback mention	Part 9
Hooks, naming convention, useState, array destructuring, multiple states, Scorecard	Part 10
Life cycle, mounting/unmounting, constructor/render/componentDidMount, strict mode double run	Part 11
Why fetch in componentDidMount; skeleton loader; UsersList; fetch/async/await; state for API data	Part 12.1 - 12.2
List rendering with map (numbers, objects), table rows, dry run, setTimeout demo	Part 12.3 - 12.4
Conditional rendering with ternary	Part 12.5
MCP discussion; human verification	Part 13
Revision material and exercises	Part 14